

ReplicaSet (RS) Notes — Beginner Friendly + Faculty Style + Interview Ready

Why ReplicaSet Came into Kubernetes?

Suppose:

- We created one POD using manifest file.
- But later that POD gets deleted.

Then:

- Application will go down.
 - No High Availability (HA).
-

Important Faculty Statement

We created a pod with manifest file,
but we deleted that pod.

If any pod deleted, it is deleted, no HA.
For HA use ReplicaSet also called RS.

What is ReplicaSet?

ReplicaSet (RS) is a Kubernetes object used to:

- Maintain multiple replicas (copies) of same POD.
-

Main Purpose of ReplicaSet

ReplicaSet is mainly used for:

- High Availability (HA)
- Self Healing
- Scaling
- Managing multiple POD replicas

Important Definition

ReplicaSet is used for managing multiple replicas of POD to perform activities like:

- Load Balancing
- Autoscaling
- High Availability

What ReplicaSet Does?

ReplicaSet:

- Creates multiple identical PODs
- Monitors PODs continuously
- Recreates POD automatically if POD fails

Important Faculty Explanation

It will create replicas of same pod.

We can use same application with multiple pods.

Even if one pod is deleted, automatically it creates another pod.

It has self-healing mechanism.

Scaling in ReplicaSet

Using ReplicaSet:

- We can increase PODs
- We can decrease PODs

Depends on requirement:

- We can scale applications.
-

Faculty Line

Depends on requirement we can scale the pods.

Flow of ReplicaSet

We create RS



RS creates PODs



RS continuously monitors PODs



If POD deleted → RS recreates POD

Real-Time Example

Suppose:

- Your application has 3 PODs.

If:

- One POD crashes

Then:

- ReplicaSet immediately creates new POD automatically.

This provides:

- High Availability
 - Self Healing
-

Labels in ReplicaSet

Why Labels Needed?

When ReplicaSet creates multiple PODs:

- All PODs have different names.

Managing them individually becomes difficult.

So:

- We use Labels to group them.
-

Faculty Explanation

As we are creating multiple pods with same application, all these pods have different names.

To group them we can give labels.

Important Point

Individual pods are difficult to manage because they have different names.

So we can give common label to group them and work together.

Example of Labels

labels:

app: bank

This means:

- All PODs belong to banking application.
-

Selector in ReplicaSet

What is Selector?

Selector is used to:

- Select PODs with same labels.
-

Faculty Definition

Selector is used to select pods with same labels.

Example

```
selector:  
  matchLabels:  
    app: bank
```

This means:

- ReplicaSet controls only PODs having label:
 app: bank
-

Important Interview Concept

What is matchLabels?

matchLabels ensures:

- Only matching labeled PODs become part of ReplicaSet.
-

Faculty Style Explanation

Ensures only pods with label app: bank
are part of this ReplicaSet.

If there is any pod with label bank,
it will become part of this ReplicaSet.

API Version for ReplicaSet

For ReplicaSet:

apiVersion: apps/v1

Important Faculty Note

For ReplicaSet use apiVersion as apps/v1

How to Find API Resources?

Command:

kubectl api-resources

This command shows:

- Available Kubernetes resources
 - API groups
 - Short names
 - Namespaced resources
-

Command to Create ReplicaSet YAML

vi replicaset.yml

Complete ReplicaSet YAML (Faculty Style)

apiVersion: apps/v1

kind: ReplicaSet

```
metadata:
  name: ib-rs
  labels:
    app: bank

spec:
  replicas: 3

selector:
  matchLabels:
    app: bank

template:

  metadata:
    labels:
      app: bank

  spec:
    containers:
      - name: cont1
        image: trainerreyaz/ib-image:latest
```

Detailed Explanation of ReplicaSet YAML

1. apiVersion

apiVersion: apps/v1

Meaning:

- Using apps API group version 1.
-

2. kind

kind: ReplicaSet

Meaning:

- Kubernetes will create ReplicaSet object.
-

3. metadata

metadata:

name: ib-rs

Meaning:

- Name of ReplicaSet is ib-rs.
-

4. labels

labels:

app: bank

Meaning:

- Grouping ReplicaSet under label:
app = bank
-

5. spec

spec:

This spec section is:

For PODS

6. replicas

replicas: 3

Meaning:

- ReplicaSet should maintain:
3 PODs always.
-

Important Faculty Note

How many number of pods

7. selector

selector:

Selector identifies:

- Which PODs belong to ReplicaSet.
-

8. matchLabels

matchLabels:

app: bank

Meaning:

- Select only PODs with label:
app: bank
-

9. template

template:

Template contains:

- Blueprint of POD.

ReplicaSet uses template to:

- Create new PODs.

Important Faculty Statement

Ensures pods get labeled as app: bank

10. template metadata

metadata:

labels:

app: bank

Assigns:

- Labels to newly created PODs.
-

11. template spec

spec:

This spec section is:

For Containers

12. containers

containers:

Contains:

- Container information.
-

13. name

- name: cont1

Container name:

- cont1
-

14. image

image: trainerreyaz/ib-image:latest

Meaning:

- Pull image from Docker registry.
-

Commands for ReplicaSet

Create ReplicaSet

```
kubectl apply -f replicaset.yml
```

Check ReplicaSets

```
kubectl get rs
```

Check PODs

```
kubectl get pods
```

Detailed Information

```
kubectl describe rs ib-rs
```

Delete POD to Test Self Healing

kubectl delete pod <pod-name>

ReplicaSet will:

- Automatically create new POD.
-

Scale ReplicaSet

Increase Replicas

kubectl scale rs ib-rs --replicas=5

Decrease Replicas

kubectl scale rs ib-rs --replicas=2

Important Interview Questions

Question:

What happens if POD is deleted in ReplicaSet?

Answer:

ReplicaSet automatically creates new POD to maintain desired replica count.

Question:

Why labels are used in ReplicaSet?

Answer:

Labels are used to:

- Group multiple PODs
 - Manage them easily
 - Allow selectors to identify correct PODs
-

Question:

What is difference between selector and labels?

Answer:

Labels	Selector
Attached to objects	Used to find objects
Used for grouping	Used for selecting

Question:

Why template section required in ReplicaSet?

Answer:

Template acts as:

- Blueprint for POD creation.

ReplicaSet uses template whenever new POD needs to be created.

Final Quick Revision

ReplicaSet maintains desired number of pods.

ReplicaSet provides:

- High Availability
- Self Healing

- Scaling

Labels are used for grouping pods.

Selectors are used for selecting pods.

Template acts as blueprint for pod creation.

ReplicaSet automatically recreates deleted pods.

Vi replicaset.yml file:

apiVersion: apps/v1

kind: ReplicaSet

metadata:

name: ib-rs

labels:

app: bank

spec:

replicas: 3

selector:

matchLabels:

app: bank

template:

metadata:

labels:

app: bank

spec:

containers:

- name: cont1

image: trainerreyaz/ib-image:latest

ReplicaSet Scaling, Update Problems & Drawbacks — Beginner Friendly Notes

Watching PODs Live

Before scaling ReplicaSet:

- Open another terminal/window.

Command:

```
kubectl get pods --watch
```

Purpose:

- Continuously monitor POD creation and deletion live.
-

List ReplicaSets

Command:

```
kubectl get rs
```

Purpose:

- Lists all ReplicaSets present in cluster.
-

Scale Out in ReplicaSet

What is Scale Out?

Scale Out means:

- Increasing number of PODs.

Example:

- 5 PODs → 10 PODs
-

Command

```
kubectl scale rs/ib-rs --replicas=10
```

Faculty Style Explanation

Now see pods creating live

Meaning:

- ReplicaSet immediately starts creating additional PODs.
-

What Happens Internally?

ReplicaSet compares:

Current replicas = 5

Desired replicas = 10

Difference:

Need 5 more PODs

So:

- ReplicaSet automatically creates 5 new PODs.
-

Scale In

What is Scale In?

Scale In means:

- Reducing number of PODs.

Example:

- 10 PODs → 5 PODs
-

Command


```
kubectl scale rs/ib-rs --replicas=5
```

Faculty Explanation

Now see pods deleting live

ReplicaSet automatically deletes extra PODs.

LIFO Concept in Scale In

Important Faculty Point

LIFO = LAST IN FIRST OUT

Meaning:

- Last created POD gets deleted first.
-

Important Definition

If a pod is created lastly,
it will delete first during scale in.

Real-Time Example

Suppose ReplicaSet created:

pod1
pod2
pod3
pod4
pod5

Later:

- Scale down to 3 replicas.

Then:

- pod5 deleted first
- pod4 deleted second

Because:

LIFO (Last In First Out)

Important Faculty Note

This scale out and scale in is manual,
later we learn how to automate.

Image Update Problem in ReplicaSet

Scenario

Suppose:

- All PODs running with image:

ib-image:latest

Now:

- We want to change image to:

mobilebanking

Important Faculty Statement

Now all pods are running with ib-image:latest image,
but if I want to change image to mobilebanking
and update the pod,
not possible in ReplicaSet.

Verify Current Image

Command:

```
kubectl describe pod -l app=bank | grep -i ID
```

Output:

- Shows currently running image IDs.
-

Editing ReplicaSet

Command:

```
kubectl edit rs/ib-rs
```

Then:

- Change image from:

internetbankingrepo

to

insurance

But Problem Happens

Even after updating:

- Existing PODs still use old image.
-

Verify Again

```
kubectl describe pod -l app=bank | grep -i ID
```

Output:

Still shows internetbanking image

Important Faculty Explanation

That's the problem with ReplicaSet.
We cannot update the application.

Why ReplicaSet Cannot Properly Roll Out Updates?

ReplicaSet mainly focuses on:

- Maintaining replica count

It does NOT properly handle:

- Application version upgrades
 - Rolling updates
 - Rollbacks
-

Major Drawback of ReplicaSet

Faculty Statement

Using ReplicaSet we cannot roll out the application.

Roll In / Roll Out Problem

Important Faculty Point

We cannot roll in and roll out,
we cannot update applications using ReplicaSet.

Let's use DEPLOYMENT.

What Happens If We Edit YAML Directly?

Suppose:

- Open replicaset.yml

Command:

```
vi replicaset.yml
```

Change image:

internetbanking → insurance

Apply again:

```
kubectl apply -f replicaset.yml
```

Problem

ReplicaSet already exists.

Possible behavior:

- Error
OR
- New PODs get new image
- Old PODs still continue with old image

This creates:

- Mixed application versions
-

Important Faculty Observation

You still see old image internetbanking.

But:

Newly created pods may contain insurance repo.

Example Scenario

Suppose:

Current 3 PODs:

POD1 → internetbanking

POD2 → internetbanking

POD3 → internetbanking

Now:

- Image changed in RS YAML.

Then:

- Scale ReplicaSet to 5.

Newly created PODs:

POD4 → insurance

POD5 → insurance

Old PODs still:

internetbanking

Important Faculty Line

Only new images are insurance.

This is drawback of ReplicaSet.

Why Deployment Came?

Because ReplicaSet cannot:

- Rollout updates properly
- Rollback versions
- Handle version management

So Kubernetes introduced:

DEPLOYMENT

Deployment internally uses:

- ReplicaSets

But provides:

- Rolling updates
 - Rollback
 - Version control
 - Zero downtime deployment
-

Advantages of ReplicaSet

Faculty Notes

Advantages:

- Self Healing
 - Scaling
-

Self Healing

If POD crashes:

- ReplicaSet creates new POD automatically.
-

Scaling

ReplicaSet allows:

- Scale Out → Increase PODs
 - Scale In → Decrease PODs
-

Drawbacks of ReplicaSet

Faculty Notes

We cannot roll in and roll out.

We cannot update applications using ReplicaSet.

Relationship Between RS and PODs

ReplicaSet → Creates & Manages PODs

Delete ReplicaSet

Command:

```
kubectl delete rs ib-rs
```

Purpose:

- Deletes ReplicaSet.
-

Important Interview Questions

Question:

What is Scale Out?

Answer:

Increasing number of PODs.

Example:

5 PODs → 10 PODs

Question:

What is Scale In?

Answer:

Reducing number of PODs.

Example:

10 PODs → 5 PODs

Question:

What is LIFO in ReplicaSet?

Answer:

Last created POD gets deleted first during scale in.

Question:

Why Deployment is preferred over ReplicaSet?

Answer:

Because ReplicaSet cannot:

- Perform rolling updates
- Rollback applications

- Properly update application versions

Deployment solves these problems.

Question:

Can ReplicaSet update existing POD images?

Answer:

No properly.

Old PODs may continue using old image while newly created PODs use new image.

Final Quick Revision

ReplicaSet provides:

- Self Healing
- Scaling
- High Availability

Scale Out:

Increase POD count

Scale In:

Decrease POD count

ReplicaSet follows:

LIFO during scale in

Major drawback:

ReplicaSet cannot properly handle application updates and rollbacks.

Deployment solves these problems.